

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №4
з дисципліни "Основи Програмної Інженерії"

Виконав
ст. гр. ПЗПІ-25-3
Коцарєв.М.О.

Прийняв:
В.Гребенюк

Харків 2026

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 4

Тема: РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Результати Code Review одного групу (Гладких.А.Р. робив code review мого коду):

№	Рядок коду	Проблема	Категорія	Рекомендація
1	8/41	Issue 1 та 4: Наявність мертвого коду	Code Smell: Dead Code	У конструктор EventSession передається registeredCount, а в метод applyForParticipation — age, проте вони ніяк не використовуються. Необхідно видалити ці параметри.
2	11	Issue 2: Порушення інкапсуляції	Code Smell: Inappropriate Intimacy / Violation of Encapsulation	Метод applyForParticipation напямую маніпулює внутрішнім станом сесії. Логіку перевірки варто перенести всередину класу EventSession.

3	32/51	Issue 3 та 5: Відсутність валідації	Code Smell: Defensive Programming	Об'єкти participant, organizer та рядок title зчитуються без попередньої перевірки на null чи undefined. Необхідно додати захисні перевірки.
4	48	Issue 6: Порушення архітектурної одноманітнос ті	Code Smell: Primitive Obsession / Inconsistency	Метод createPaidTou r повертає звичайний літерал об'єкта. Необхідно створити повноцінний клас PaidTour та повертати його екземпляр.

Посилання на PR з коментарями Code Review : <https://github.com/mik-kotsariev/DeltaPhys/pull/25>

Результати рефакторингу власного коду: <https://github.com/mik-kotsariev/DeltaPhys/pull/23>

Операція 1: Відновлення інкапсуляції

- **БУЛО:** `if (session.registeredUsers.has(pId))` та `session.registeredUsers.add(pId);`
- **СТАЛО:** `session.registerParticipant(pId);` (вся логіка перевірки перенесена всередину класу `EventSession`).
- **ЧОМУ:** Об'єкт має самостійно керувати своїм станом. Пряме звернення до властивостей іншого об'єкта порушує принципи ООП (Закон Деметри) і робить код крихким у разі зміни структури даних (наприклад, із `Set` на `Array`).

Операція 2: Видалення мертвого коду

- **БУЛО:** `constructor(id, title, registeredCount, maxCapacity)` та `applyForParticipation(participant, session, age).`
- **СТАЛО:** `constructor(id, title, maxCapacity)` та `applyForParticipation(participant, session).`
- **ЧОМУ:** Видалення невикористаних параметрів робить сигнатуру функцій чистішою та запобігає плутанині під час виклику методів іншими розробниками.

Операція 3: Додавання захисних перевірок (Guard Clauses / Fail Fast)

- **БУЛО:** `const pId = participant.id;` та `const trimmedTitle = title.trim();` (без перевірок).
- **СТАЛО:** `if (!participant || typeof participant !== 'object' || !(id in participant))` `throw new TypeError("...");` перед зчитуванням властивостей.
- **ЧОМУ:** Запобігання `Runtime Error` під час виконання. Патерн `Fail Fast` гарантує, що некоректні дані перехоплюються на самому початку функції, видаючи чітку помилку.

Звіт регресійного тестування:

```
✓ 9. Негативний (EP): Учасник без id (undefined) -> помилка при другій реєстрації
Method: createPaidTour (Edge Cases & Type Coercion)
✓ 10. Негативний (EP): Назва передана як число (123) -> помилка (TypeError)
✓ 11. Негативний (EP): Назва передана як об'єкт ({} ) -> помилка (TypeError)
✓ 12. Негативний (BVA): Назва містить лише символ нового рядка (\n) -> помилка (1 ms)
✓ 13. Позитивний (EP): Успішне створення туру

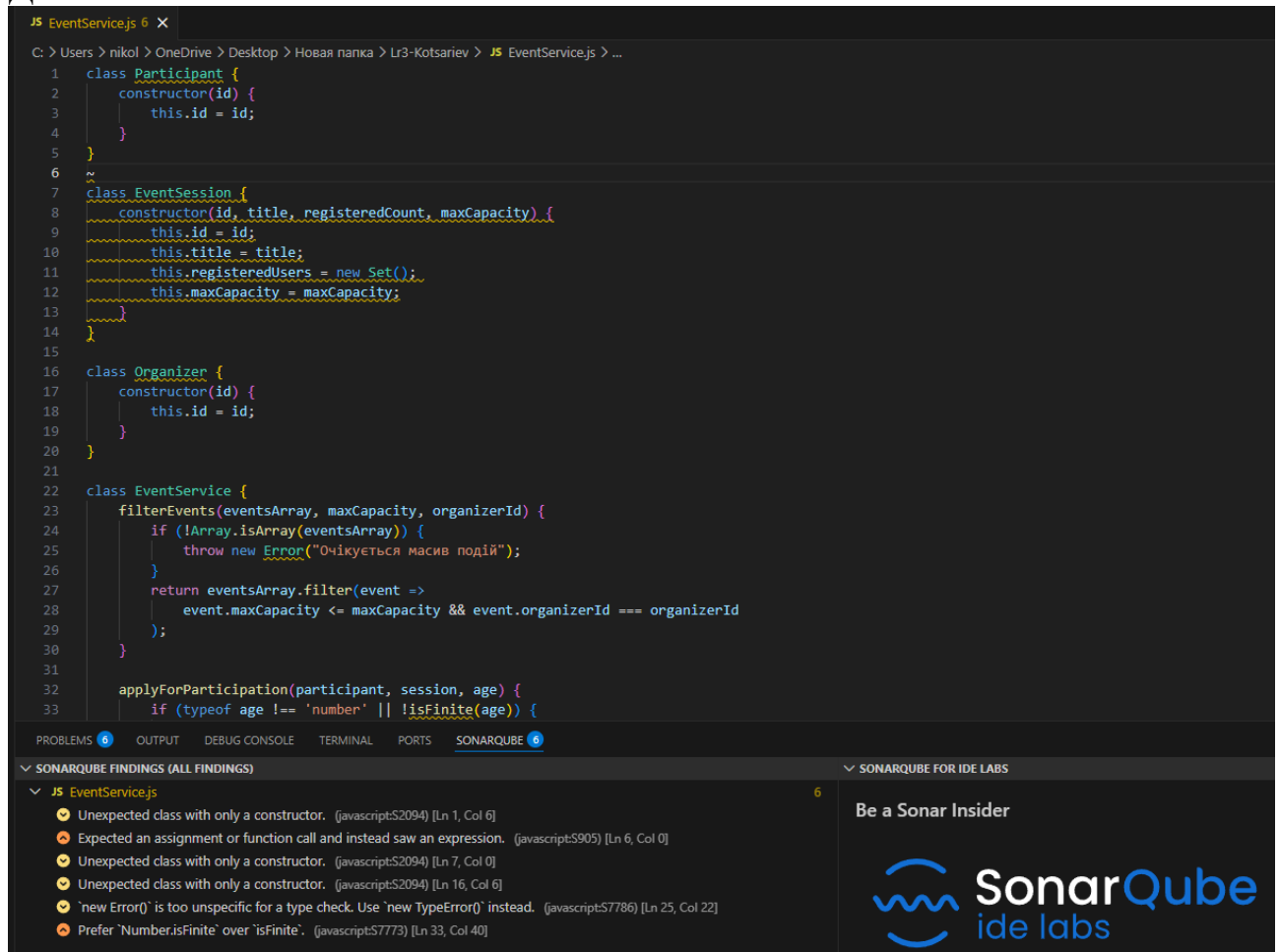
-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files |    94.11 |      92 |     100 |    94.11 |
EventService.js |    94.11 |      92 |     100 |    94.11 | 56,72
-----|-----|-----|-----|-----|-----

Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        0.349 s, estimated 1 s
Ran all test suites.
```

Порівняння метрик «ДО / ПІСЛЯ» :

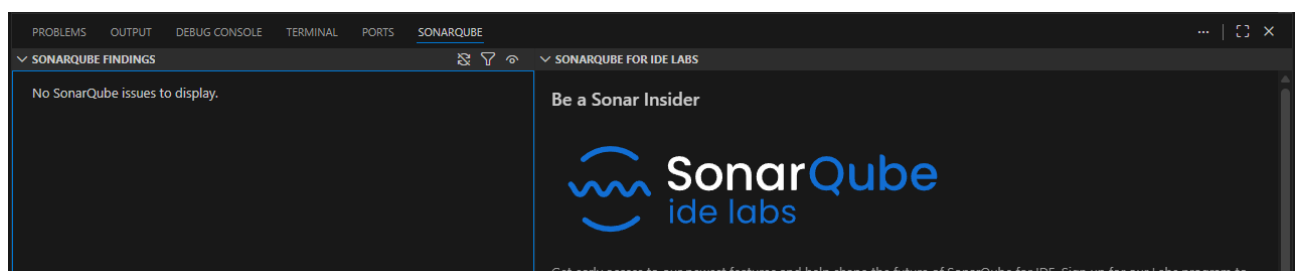
SonarLint issues:

ДО:



The screenshot shows a VS Code editor with a file named `EventService.js`. The code contains several JavaScript classes: `Participant`, `EventSession`, `Organizer`, and `EventService`. The `EventService` class has two methods: `filterEvents` and `applyForParticipation`. Below the editor, the SonarLint interface is visible, showing a list of findings under the heading "SONARQUBE FINDINGS (ALL FINDINGS)". There are six findings listed, all related to the `EventService.js` file. The findings include: "Unexpected class with only a constructor." (three instances), "Expected an assignment or function call and instead saw an expression." (one instance), "'new Error()' is too unspecific for a type check. Use 'new TypeError()' instead." (one instance), and "Prefer 'Number.isFinite' over 'isFinite'." (one instance). On the right side of the SonarLint panel, there is a "Be a Sonar Insider" banner with the SonarQube logo and a link to the SonarQube IDE Labs program.

ПІСЛЯ:



The screenshot shows the same VS Code editor with the `EventService.js` file. The SonarLint interface is still visible, but the "SONARQUBE FINDINGS" panel now displays "No SonarQube issues to display." The "Be a Sonar Insider" banner is still present on the right side of the panel.

Цикломатична складність (за допомогою Lizard):

ДО:

```
PS C:\Users\nikol\OneDrive\Desktop\Новая папка\Lr3-Kotsariev> lizard EventService.js
=====
NLOC   CCN   token  PARAM  length  location
-----
      3     1    11     1      3  constructor@2-4@EventService.js
      6     1    38     4      6  constructor@8-13@EventService.js
      3     1    11     1      3  constructor@17-19@EventService.js
      1     1     1     0      1  (anonymous)@28-28@EventService.js
      8     3    48     3      8  filterEvents@23-30@EventService.js
     14     7    96     3     18  applyForParticipation@32-49@EventService.js
     12     2    56     4     14  createPaidTour@51-64@EventService.js
1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     56       6.7    2.3      37.3         7  EventService.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc   Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     56       6.7    2.3      37.3         7         0    0.00  0.00

PS C:\Users\nikol\OneDrive\Desktop\Новая папка\Lr3-Kotsariev>
```

ПІСЛЯ:

```
C:\Users\nikol\OneDrive\Desktop\Lr3-Kotsariev>lizard EventService.js
=====
NLOC   CCN   token  PARAM  length  location
-----
      3     1    11     1      3  constructor@2-4@EventService.js
      6     1    36     3      6  constructor@9-14@EventService.js
      3     1    15     1      3  hasParticipant@16-18@EventService.js
      3     1    14     1      3  registerParticipant@20-22@EventService.js
      3     1    11     1      3  constructor@26-28@EventService.js
      6     1    35     4      6  constructor@32-37@EventService.js
      1     1     1     0      1  (anonymous)@46-46@EventService.js
      8     3    48     3      8  filterEvents@41-48@EventService.js
     14     8    94     2     18  applyForParticipation@50-67@EventService.js
     13     6    92     4     17  createPaidTour@69-85@EventService.js
1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     70       6.0    2.4      35.7        10  EventService.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc   Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     70       6.0    2.4      35.7        10         0    0.00  0.00
```

ВИСНОВОК

В ході виконання лабораторної роботи було успішно закріплено практичні навички проведення Code Review та рефакторингу коду за сучасними індустріальними стандартами.

Під час взаємного рецензування коду (через GitHub Pull Requests) було ідентифіковано та класифіковано низку архітектурних проблем та типових «запахів коду» (code smells), зокрема: порушення принципів інкапсуляції, наявність мертвого коду, відсутність захисних перевірок (Defensive Programming) та використання базових примітивів замість об'єктів предметної області.

На основі отриманих зауважень було проведено серію рефакторинг-операцій. Перенесення логіки всередину відповідних класів (відновлення інкапсуляції),

використання патерну Fail Fast (Guard Clauses) для валідації вхідних даних та очищення сигнатур методів дозволили зробити код більш безпечним, масштабованим та читабельним.

Критично важливим етапом стала верифікація внесених змін за допомогою регресійного тестування (фреймворк Jest). Успішне проходження всіх модульних тестів підтвердило, що рефакторинг зберіг початкову поведінку системи. Крім того, інструментальний статичний аналіз (SonarLint та утиліта Lizard) документально зафіксував покращення якості коду: усунення потенційних вразливостей та оптимізацію цикломатичної складності методів. Загалом, робота наочно продемонструвала, що регулярний Code Review у поєднанні з рефакторингом та покриттям коду тестами є невіддільною складовою процесу розробки надійного та якісного програмного забезпечення.

Підсумкова рефлексія Під час виконання цієї лабораторної роботи я переконався, що Code Review — це потужний інструмент не лише для пошуку багів, але й для покращення архітектурної якості коду. Рефакторинг за принципами ООП суттєво покращив архітектуру мого застосунку: приховування структури Set всередині EventSession (інкапсуляція), видалення мертвого коду та використання Guard Clauses зробили код значно чистішим та безпечнішим. Також я усвідомив критичну важливість регресійного тестування: наявність 14 модульних тестів (Jest) дозволила мені впевнено переписувати внутрішню логіку сервісів, знаючи, що жодна попередня функціональність не була зламана.